

BACKEND

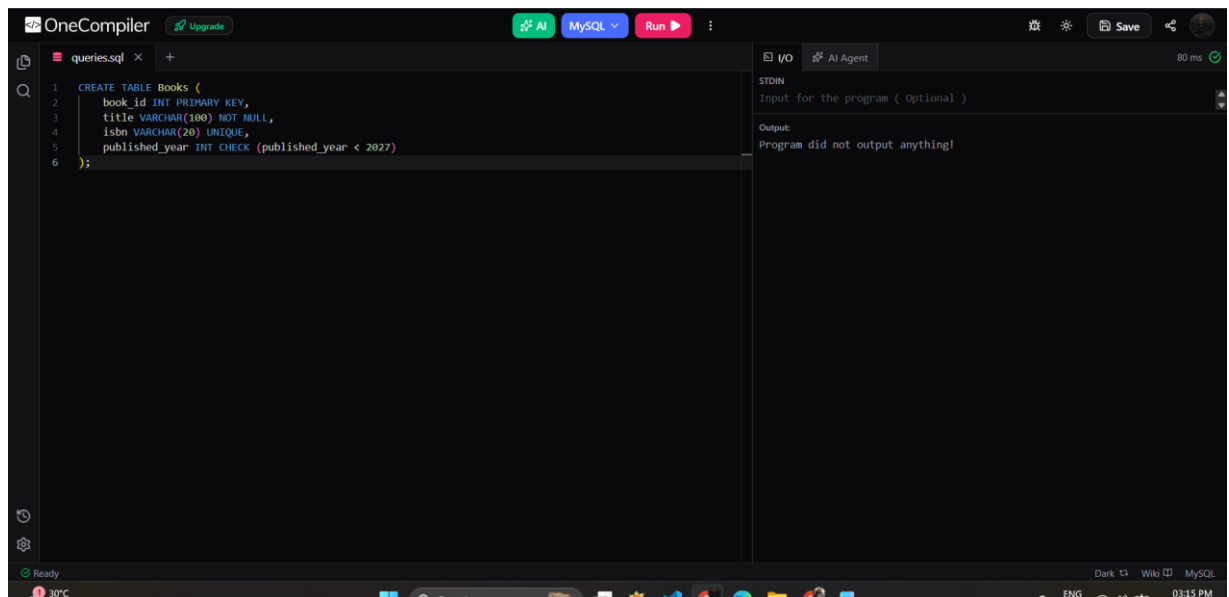
WEEK 2

K ANUSRI
2520030432
SEC 7

1. CREATE DATABASE

```
CREATE DATABASE bookflow_db;  
USE bookflow_db;
```

2. Create Books Table

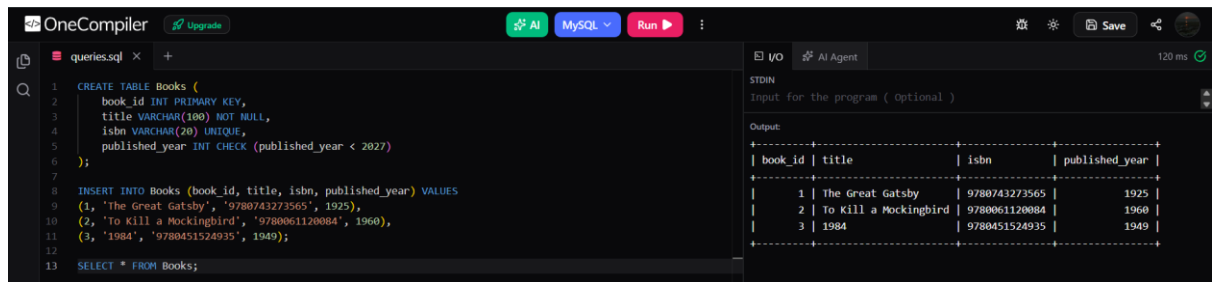


The screenshot shows the OneCompiler IDE interface. The main editor window displays a SQL query to create a table named 'Books'. The query is as follows:

```
1 CREATE TABLE Books (  
2   book_id INT PRIMARY KEY,  
3   title VARCHAR(100) NOT NULL,  
4   isbn VARCHAR(20) UNIQUE,  
5   published_year INT CHECK (published_year < 2027)  
6 );
```

The right-hand panel shows the 'I/O' section with 'STDIN' and 'Output' tabs. The 'STDIN' tab contains the text 'Input for the program (optional)'. The 'Output' tab contains the text 'Program did not output anything!'. The top bar of the IDE shows 'OneCompiler', 'Upgrade', 'AI', 'MySQL', 'Run', and 'Save' buttons. The bottom status bar shows 'Ready', '30°C', 'Dark', 'Wiki', 'MySQL', and '03:15 PM'.

3. Insert Books and Display Books Table

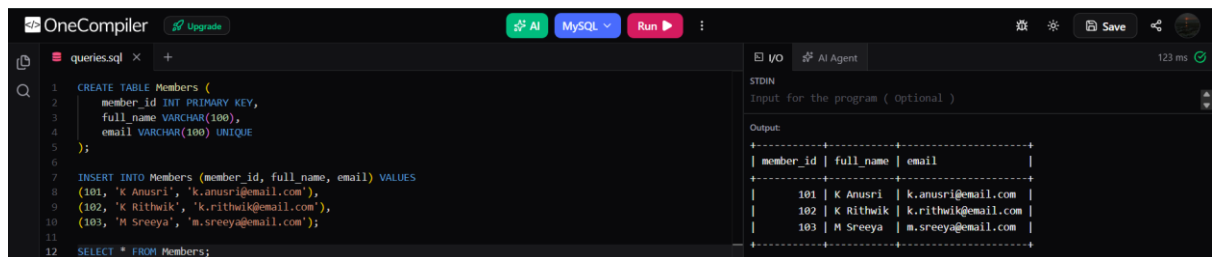


```
1 CREATE TABLE Books (  
2     book_id INT PRIMARY KEY,  
3     title VARCHAR(100) NOT NULL,  
4     isbn VARCHAR(20) UNIQUE,  
5     published_year INT CHECK (published_year < 2027)  
6 );  
7  
8 INSERT INTO Books (book_id, title, isbn, published_year) VALUES  
9 (1, 'The Great Gatsby', '9780743273565', 1925),  
10 (2, 'To Kill a Mockingbird', '9780061120084', 1960),  
11 (3, '1984', '9780451524935', 1949);  
12  
13 SELECT * FROM Books;
```

Output:

book_id	title	isbn	published_year
1	The Great Gatsby	9780743273565	1925
2	To Kill a Mockingbird	9780061120084	1960
3	1984	9780451524935	1949

4. Create, Insert and Display Members Table

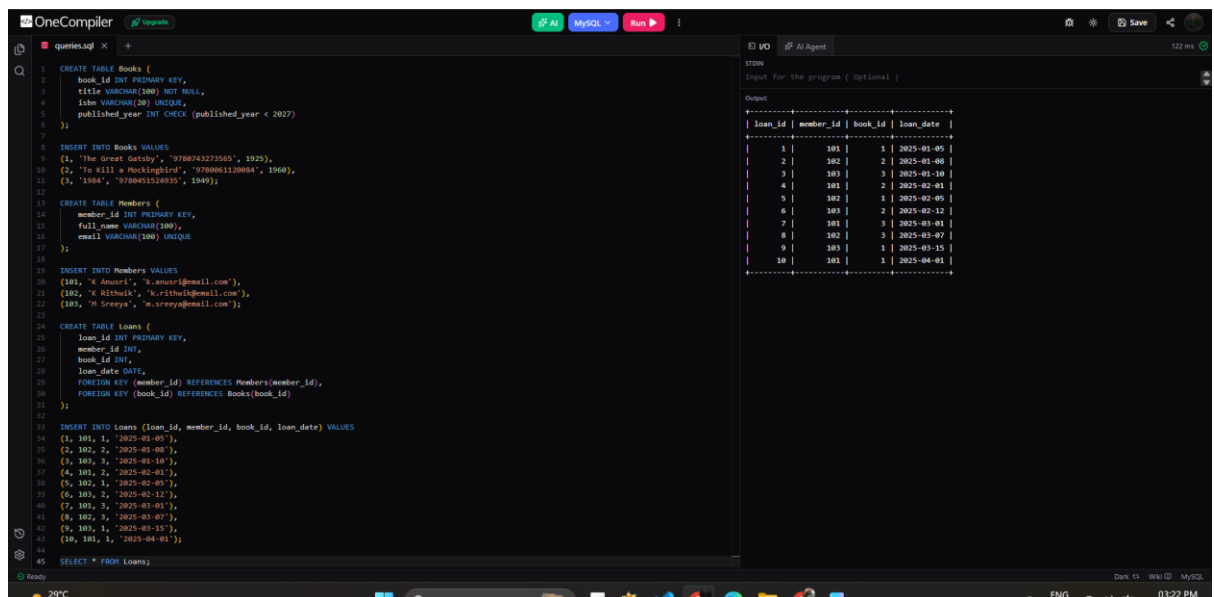


```
1 CREATE TABLE Members (  
2     member_id INT PRIMARY KEY,  
3     full_name VARCHAR(100),  
4     email VARCHAR(100) UNIQUE  
5 );  
6  
7 INSERT INTO Members (member_id, full_name, email) VALUES  
8 (101, 'K Anusri', 'k.anusri@gmail.com'),  
9 (102, 'K Rithwik', 'k.rithwik@gmail.com'),  
10 (103, 'M Sreeya', 'm.sreeya@gmail.com');  
11  
12 SELECT * FROM Members;
```

Output:

member_id	full_name	email
101	K Anusri	k.anusri@gmail.com
102	K Rithwik	k.rithwik@gmail.com
103	M Sreeya	m.sreeya@gmail.com

5. Create, Insert and Display Loans Table

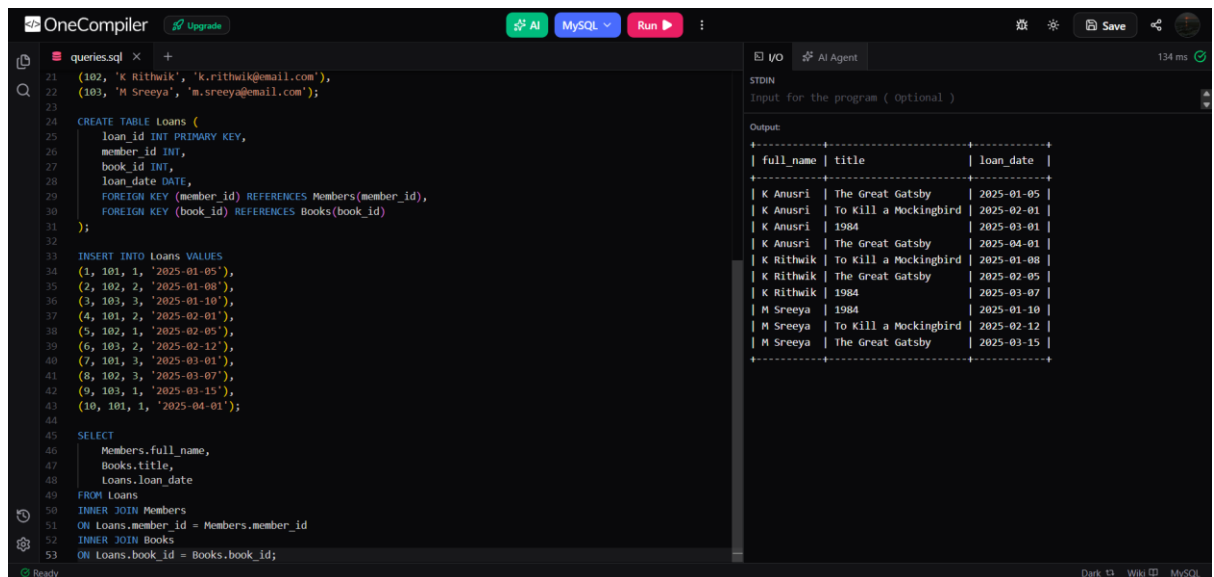


```
1 CREATE TABLE Books (  
2     book_id INT PRIMARY KEY,  
3     title VARCHAR(100) NOT NULL,  
4     isbn VARCHAR(20) UNIQUE,  
5     published_year INT CHECK (published_year < 2027)  
6 );  
7  
8 INSERT INTO Books VALUES  
9 (1, 'The Great Gatsby', '9780743273565', 1925),  
10 (2, 'To Kill a Mockingbird', '9780061120084', 1960),  
11 (3, '1984', '9780451524935', 1949);  
12  
13 CREATE TABLE Members (  
14     member_id INT PRIMARY KEY,  
15     full_name VARCHAR(100),  
16     email VARCHAR(100) UNIQUE  
17 );  
18  
19 INSERT INTO Members VALUES  
20 (101, 'K Anusri', 'k.anusri@gmail.com'),  
21 (102, 'K Rithwik', 'k.rithwik@gmail.com'),  
22 (103, 'M Sreeya', 'm.sreeya@gmail.com');  
23  
24 CREATE TABLE Loans (  
25     loan_id INT PRIMARY KEY,  
26     member_id INT,  
27     book_id INT,  
28     loan_date DATE,  
29     FOREIGN KEY (member_id) REFERENCES Members(member_id),  
30     FOREIGN KEY (book_id) REFERENCES Books(book_id)  
31 );  
32  
33 INSERT INTO Loans (loan_id, member_id, book_id, loan_date) VALUES  
34 (1, 101, 1, '2025-01-05'),  
35 (2, 102, 2, '2025-01-08'),  
36 (3, 103, 3, '2025-01-10'),  
37 (4, 101, 2, '2025-02-01'),  
38 (5, 102, 1, '2025-02-05'),  
39 (6, 103, 2, '2025-02-12'),  
40 (7, 101, 3, '2025-03-01'),  
41 (8, 102, 1, '2025-03-07'),  
42 (9, 103, 1, '2025-03-15'),  
43 (10, 101, 1, '2025-04-01');  
44  
45 SELECT * FROM Loans;
```

Output:

loan_id	member_id	book_id	loan_date
1	101	1	2025-01-05
2	102	2	2025-01-08
3	103	3	2025-01-10
4	101	2	2025-02-01
5	102	1	2025-02-05
6	103	2	2025-02-12
7	101	3	2025-03-01
8	102	3	2025-03-07
9	103	1	2025-03-15
10	101	1	2025-04-01

6. Display Borrowed Books Using INNER JOIN

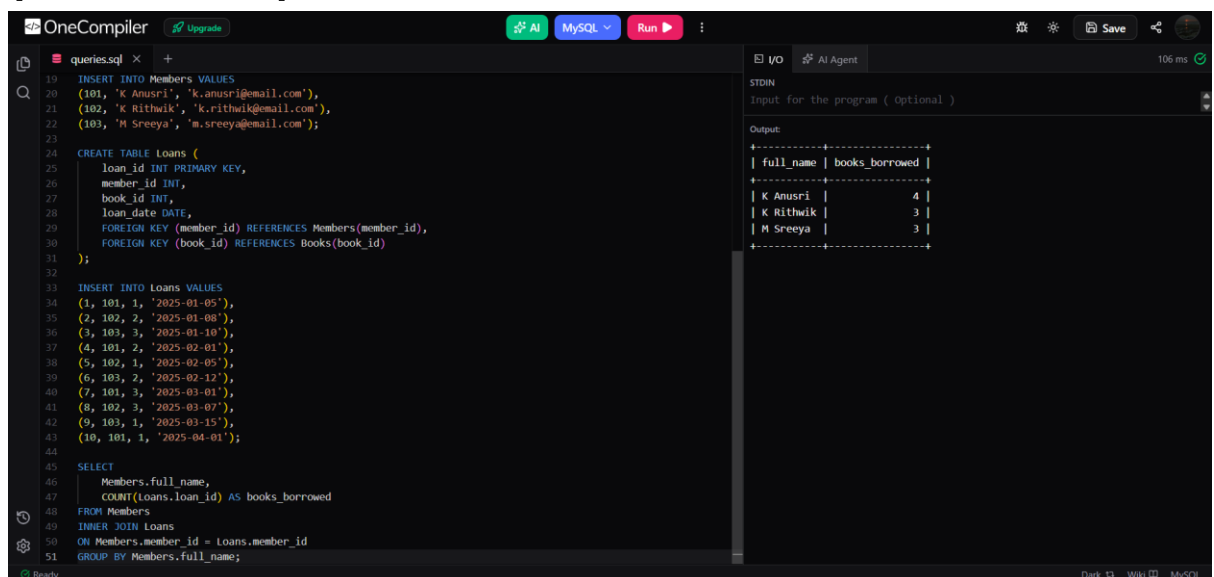


```
queries.sql x +
21 (102, 'K Rithwik', 'k.rithwik@email.com'),
22 (103, 'M Sreeya', 'm.sreeya@email.com');
23
24 CREATE TABLE Loans (
25     loan_id INT PRIMARY KEY,
26     member_id INT,
27     book_id INT,
28     loan_date DATE,
29     FOREIGN KEY (member_id) REFERENCES Members(member_id),
30     FOREIGN KEY (book_id) REFERENCES Books(book_id)
31 );
32
33 INSERT INTO Loans VALUES
34 (1, 101, 1, '2025-01-05'),
35 (2, 102, 2, '2025-01-08'),
36 (3, 103, 3, '2025-01-10'),
37 (4, 101, 2, '2025-02-01'),
38 (5, 102, 1, '2025-02-05'),
39 (6, 103, 2, '2025-02-12'),
40 (7, 101, 3, '2025-03-01'),
41 (8, 102, 3, '2025-03-07'),
42 (9, 103, 1, '2025-03-15'),
43 (10, 101, 1, '2025-04-01');
44
45 SELECT
46     Members.full_name,
47     Books.title,
48     Loans.loan_date
49 FROM Loans
50 INNER JOIN Members
51 ON Loans.member_id = Members.member_id
52 INNER JOIN Books
53 ON Loans.book_id = Books.book_id;
```

Output:

full_name	title	loan_date
K Anusri	The Great Gatsby	2025-01-05
K Anusri	To Kill a Mockingbird	2025-02-01
K Anusri	1984	2025-03-01
K Anusri	The Great Gatsby	2025-04-01
K Rithwik	To Kill a Mockingbird	2025-01-08
K Rithwik	The Great Gatsby	2025-02-05
K Rithwik	1984	2025-03-07
M Sreeya	1984	2025-01-10
M Sreeya	To Kill a Mockingbird	2025-02-12
M Sreeya	The Great Gatsby	2025-03-15

7. Count Books Borrowed by Each Member (GROUP BY)

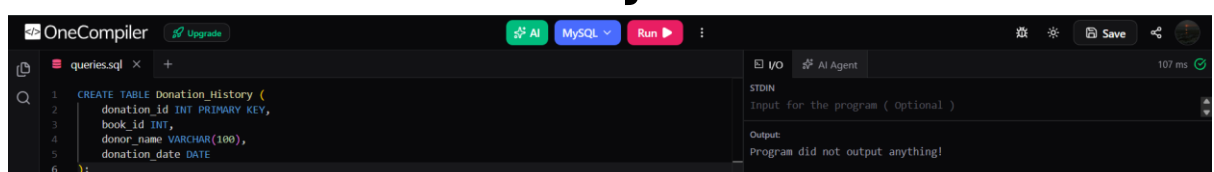


```
queries.sql x +
19 INSERT INTO Members VALUES
20 (101, 'K Anusri', 'k.anusri@email.com'),
21 (102, 'K Rithwik', 'k.rithwik@email.com'),
22 (103, 'M Sreeya', 'm.sreeya@email.com');
23
24 CREATE TABLE Loans (
25     loan_id INT PRIMARY KEY,
26     member_id INT,
27     book_id INT,
28     loan_date DATE,
29     FOREIGN KEY (member_id) REFERENCES Members(member_id),
30     FOREIGN KEY (book_id) REFERENCES Books(book_id)
31 );
32
33 INSERT INTO Loans VALUES
34 (1, 101, 1, '2025-01-05'),
35 (2, 102, 2, '2025-01-08'),
36 (3, 103, 3, '2025-01-10'),
37 (4, 101, 2, '2025-02-01'),
38 (5, 102, 1, '2025-02-05'),
39 (6, 103, 2, '2025-02-12'),
40 (7, 101, 3, '2025-03-01'),
41 (8, 102, 3, '2025-03-07'),
42 (9, 103, 1, '2025-03-15'),
43 (10, 101, 1, '2025-04-01');
44
45 SELECT
46     Members.full_name,
47     COUNT(Loans.loan_id) AS books_borrowed
48 FROM Members
49 INNER JOIN Loans
50 ON Members.member_id = Loans.member_id
51 GROUP BY Members.full_name;
```

Output:

full_name	books_borrowed
K Anusri	4
K Rithwik	3
M Sreeya	3

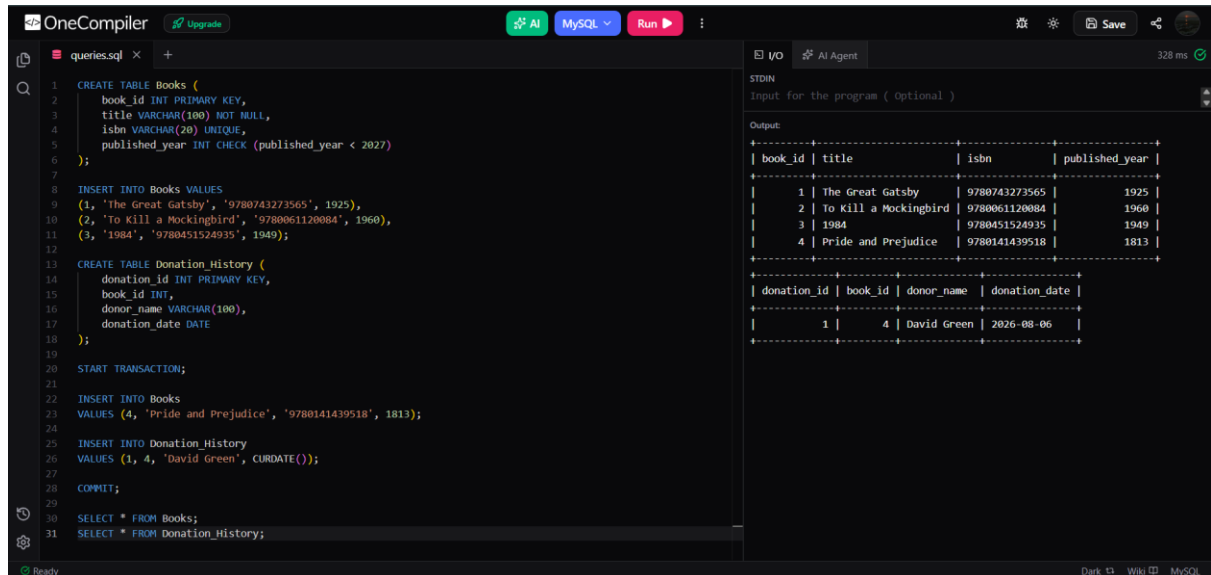
8. Create Donation History Table



```
queries.sql x +
1 CREATE TABLE Donation_History (
2     donation_id INT PRIMARY KEY,
3     book_id INT,
4     donor_name VARCHAR(100),
5     donation_date DATE
6 );
```

Output: Program did not output anything!

9. Perform Transaction Using START TRANSACTION and COMMIT



The screenshot shows the OneCompiler MySQL IDE. The left pane contains a SQL script with the following queries:

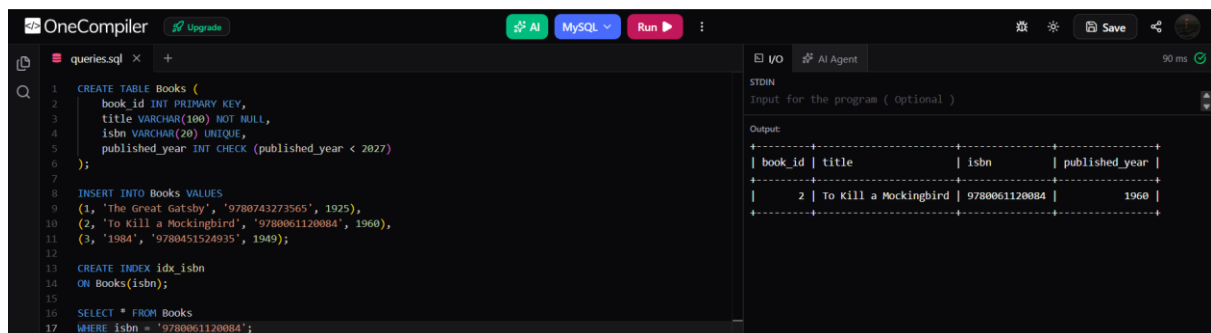
```
1 CREATE TABLE Books (  
2   book_id INT PRIMARY KEY,  
3   title VARCHAR(100) NOT NULL,  
4   isbn VARCHAR(20) UNIQUE,  
5   published_year INT CHECK (published_year < 2027)  
6 );  
7  
8 INSERT INTO Books VALUES  
9 (1, 'The Great Gatsby', '9780743273565', 1925),  
10 (2, 'To Kill a Mockingbird', '9780061120084', 1960),  
11 (3, '1984', '9780451524935', 1949);  
12  
13 CREATE TABLE Donation_History (  
14   donation_id INT PRIMARY KEY,  
15   book_id INT,  
16   donor_name VARCHAR(100),  
17   donation_date DATE  
18 );  
19  
20 START TRANSACTION;  
21  
22 INSERT INTO Books  
23 VALUES (4, 'Pride and Prejudice', '9780141439518', 1813);  
24  
25 INSERT INTO Donation_History  
26 VALUES (1, 4, 'David Green', CURDATE());  
27  
28 COMMIT;  
29  
30 SELECT * FROM Books;  
31 SELECT * FROM Donation_History;
```

The right pane shows the output of the queries, which is displayed in two tables:

book_id	title	isbn	published_year
1	The Great Gatsby	9780743273565	1925
2	To Kill a Mockingbird	9780061120084	1960
3	1984	9780451524935	1949
4	Pride and Prejudice	9780141439518	1813

donation_id	book_id	donor_name	donation_date
1	4	David Green	2026-08-06

10. Create Index and Search Book by ISBN



The screenshot shows the OneCompiler MySQL IDE. The left pane contains a SQL script with the following queries:

```
1 CREATE TABLE Books (  
2   book_id INT PRIMARY KEY,  
3   title VARCHAR(100) NOT NULL,  
4   isbn VARCHAR(20) UNIQUE,  
5   published_year INT CHECK (published_year < 2027)  
6 );  
7  
8 INSERT INTO Books VALUES  
9 (1, 'The Great Gatsby', '9780743273565', 1925),  
10 (2, 'To Kill a Mockingbird', '9780061120084', 1960),  
11 (3, '1984', '9780451524935', 1949);  
12  
13 CREATE INDEX idx_isbn  
14 ON Books(isbn);  
15  
16 SELECT * FROM Books  
17 WHERE isbn = '9780061120084';
```

The right pane shows the output of the queries, which is displayed in a single table:

book_id	title	isbn	published_year
2	To Kill a Mockingbird	9780061120084	1960

ASSIGNMENT 1

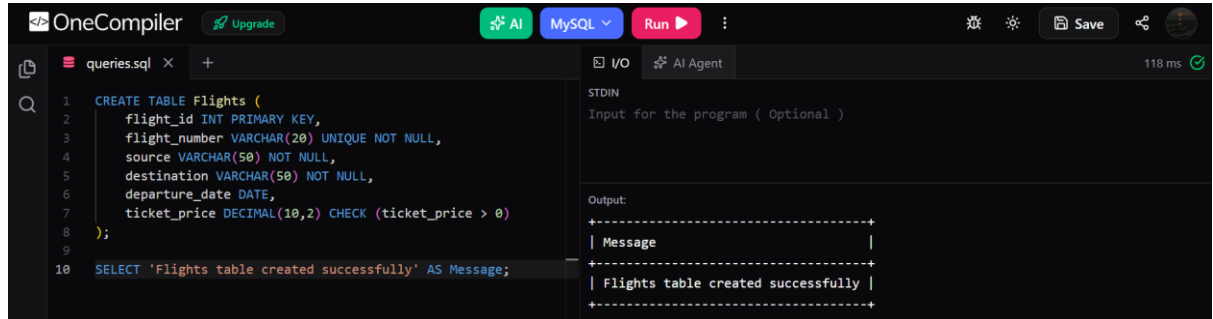
1. Create Database

```
CREATE DATABASE SkyTrack_DB;  
USE SkyTrack_DB;
```

Expected Output:

Database changed

2. Create Flights Table



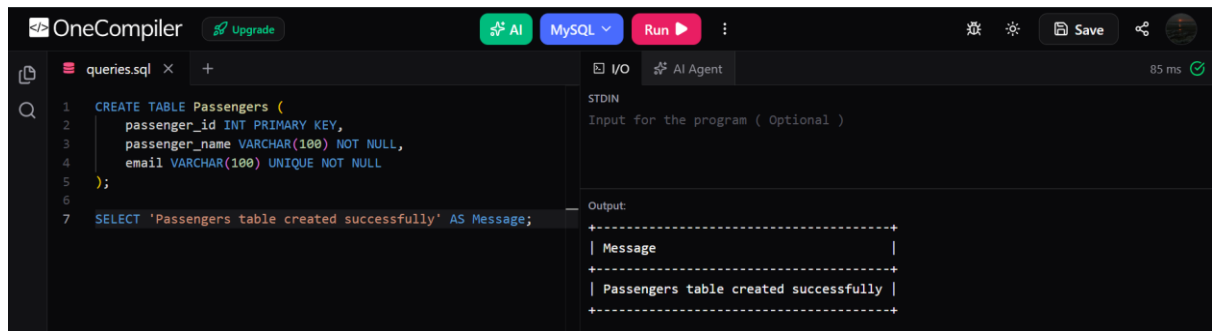
The screenshot shows the OneCompiler IDE with a MySQL database. The code in the editor creates a table named 'Flights' with columns: flight_id (INT PRIMARY KEY), flight_number (VARCHAR(20) UNIQUE NOT NULL), source (VARCHAR(50) NOT NULL), destination (VARCHAR(50) NOT NULL), departure_date (DATE), and ticket_price (DECIMAL(10,2) CHECK (ticket_price > 0)). A message is also printed: 'Flights table created successfully'. The output pane shows the message: 'Flights table created successfully'.

```
1 CREATE TABLE Flights (  
2   flight_id INT PRIMARY KEY,  
3   flight_number VARCHAR(20) UNIQUE NOT NULL,  
4   source VARCHAR(50) NOT NULL,  
5   destination VARCHAR(50) NOT NULL,  
6   departure_date DATE,  
7   ticket_price DECIMAL(10,2) CHECK (ticket_price > 0)  
8 );  
9  
10 SELECT 'Flights table created successfully' AS Message;
```

Output:

```
+-----+  
| Message |  
+-----+  
| Flights table created successfully |  
+-----+
```

3. Create Passengers Table



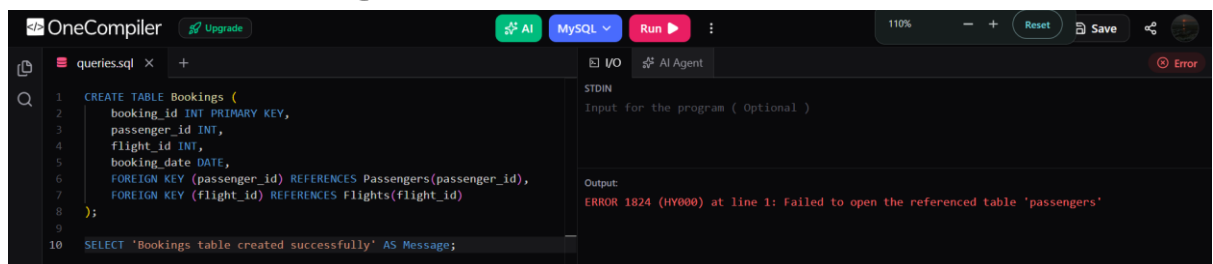
The screenshot shows the OneCompiler IDE with a MySQL database. The code in the editor creates a table named 'Passengers' with columns: passenger_id (INT PRIMARY KEY), passenger_name (VARCHAR(100) NOT NULL), and email (VARCHAR(100) UNIQUE NOT NULL). A message is also printed: 'Passengers table created successfully'. The output pane shows the message: 'Passengers table created successfully'.

```
1 CREATE TABLE Passengers (  
2   passenger_id INT PRIMARY KEY,  
3   passenger_name VARCHAR(100) NOT NULL,  
4   email VARCHAR(100) UNIQUE NOT NULL  
5 );  
6  
7 SELECT 'Passengers table created successfully' AS Message;
```

Output:

```
+-----+  
| Message |  
+-----+  
| Passengers table created successfully |  
+-----+
```

4. Create Bookings Table



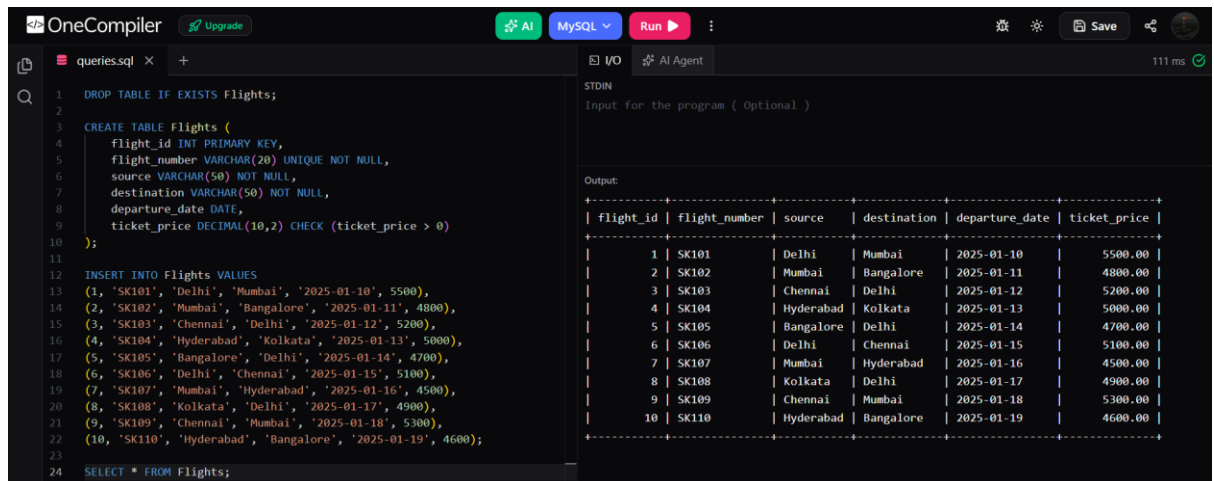
The screenshot shows the OneCompiler IDE with a MySQL database. The code in the editor creates a table named 'Bookings' with columns: booking_id (INT PRIMARY KEY), passenger_id (INT), flight_id (INT), booking_date (DATE), and foreign keys to Passengers (passenger_id) and Flights (flight_id). A message is also printed: 'Bookings table created successfully'. The output pane shows an error: 'ERROR 1824 (HY000) at line 1: Failed to open the referenced table 'passengers''.

```
1 CREATE TABLE Bookings (  
2   booking_id INT PRIMARY KEY,  
3   passenger_id INT,  
4   flight_id INT,  
5   booking_date DATE,  
6   FOREIGN KEY (passenger_id) REFERENCES Passengers(passenger_id),  
7   FOREIGN KEY (flight_id) REFERENCES Flights(flight_id)  
8 );  
9  
10 SELECT 'Bookings table created successfully' AS Message;
```

Output:

```
ERROR 1824 (HY000) at line 1: Failed to open the referenced table 'passengers'
```

5. Insert Flight Records



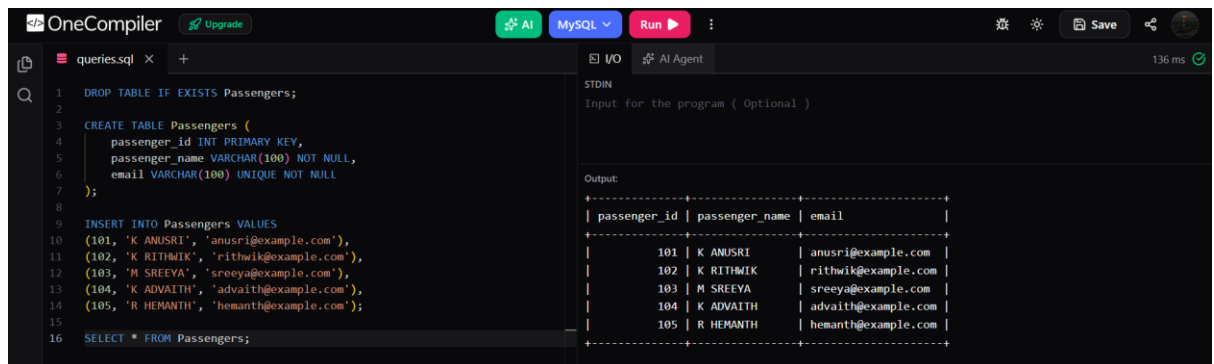
The screenshot shows the OneCompiler MySQL interface. The left pane contains the following SQL code:

```
1 DROP TABLE IF EXISTS Flights;
2
3 CREATE TABLE Flights (
4     flight_id INT PRIMARY KEY,
5     flight_number VARCHAR(20) UNIQUE NOT NULL,
6     source VARCHAR(50) NOT NULL,
7     destination VARCHAR(50) NOT NULL,
8     departure_date DATE,
9     ticket_price DECIMAL(10,2) CHECK (ticket_price > 0)
10 );
11
12 INSERT INTO Flights VALUES
13 (1, 'SK101', 'Delhi', 'Mumbai', '2025-01-10', 5500),
14 (2, 'SK102', 'Mumbai', 'Bangalore', '2025-01-11', 4800),
15 (3, 'SK103', 'Chennai', 'Delhi', '2025-01-12', 5200),
16 (4, 'SK104', 'Hyderabad', 'Kolkata', '2025-01-13', 5000),
17 (5, 'SK105', 'Bangalore', 'Delhi', '2025-01-14', 4700),
18 (6, 'SK106', 'Delhi', 'Chennai', '2025-01-15', 5100),
19 (7, 'SK107', 'Mumbai', 'Hyderabad', '2025-01-16', 4500),
20 (8, 'SK108', 'Kolkata', 'Delhi', '2025-01-17', 4900),
21 (9, 'SK109', 'Chennai', 'Mumbai', '2025-01-18', 5300),
22 (10, 'SK110', 'Hyderabad', 'Bangalore', '2025-01-19', 4600);
23
24 SELECT * FROM Flights;
```

The right pane shows the output of the query, displaying a table with 10 rows of flight records:

flight_id	flight_number	source	destination	departure_date	ticket_price
1	SK101	Delhi	Mumbai	2025-01-10	5500.00
2	SK102	Mumbai	Bangalore	2025-01-11	4800.00
3	SK103	Chennai	Delhi	2025-01-12	5200.00
4	SK104	Hyderabad	Kolkata	2025-01-13	5000.00
5	SK105	Bangalore	Delhi	2025-01-14	4700.00
6	SK106	Delhi	Chennai	2025-01-15	5100.00
7	SK107	Mumbai	Hyderabad	2025-01-16	4500.00
8	SK108	Kolkata	Delhi	2025-01-17	4900.00
9	SK109	Chennai	Mumbai	2025-01-18	5300.00
10	SK110	Hyderabad	Bangalore	2025-01-19	4600.00

6. Insert Passenger Records



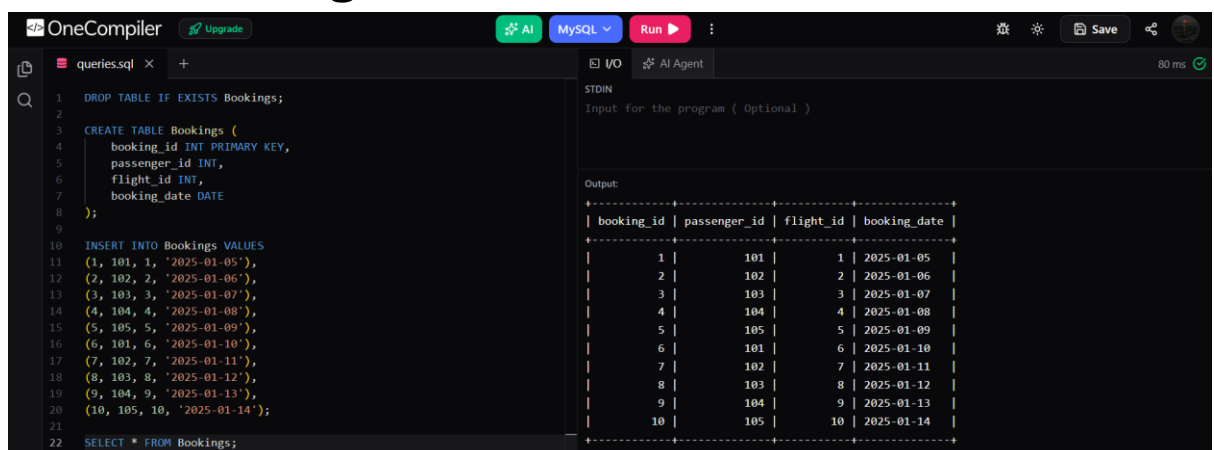
The screenshot shows the OneCompiler MySQL interface. The left pane contains the following SQL code:

```
1 DROP TABLE IF EXISTS Passengers;
2
3 CREATE TABLE Passengers (
4     passenger_id INT PRIMARY KEY,
5     passenger_name VARCHAR(100) NOT NULL,
6     email VARCHAR(100) UNIQUE NOT NULL
7 );
8
9 INSERT INTO Passengers VALUES
10 (101, 'K ANUSRI', 'anusri@example.com'),
11 (102, 'K RITHWIK', 'rithwik@example.com'),
12 (103, 'M SREEYA', 'sreeya@example.com'),
13 (104, 'K ADVAITH', 'advait@example.com'),
14 (105, 'R HEMANTH', 'hemanth@example.com');
15
16 SELECT * FROM Passengers;
```

The right pane shows the output of the query, displaying a table with 5 rows of passenger records:

passenger_id	passenger_name	email
101	K ANUSRI	anusri@example.com
102	K RITHWIK	rithwik@example.com
103	M SREEYA	sreeya@example.com
104	K ADVAITH	advait@example.com
105	R HEMANTH	hemanth@example.com

7. Insert Booking Records



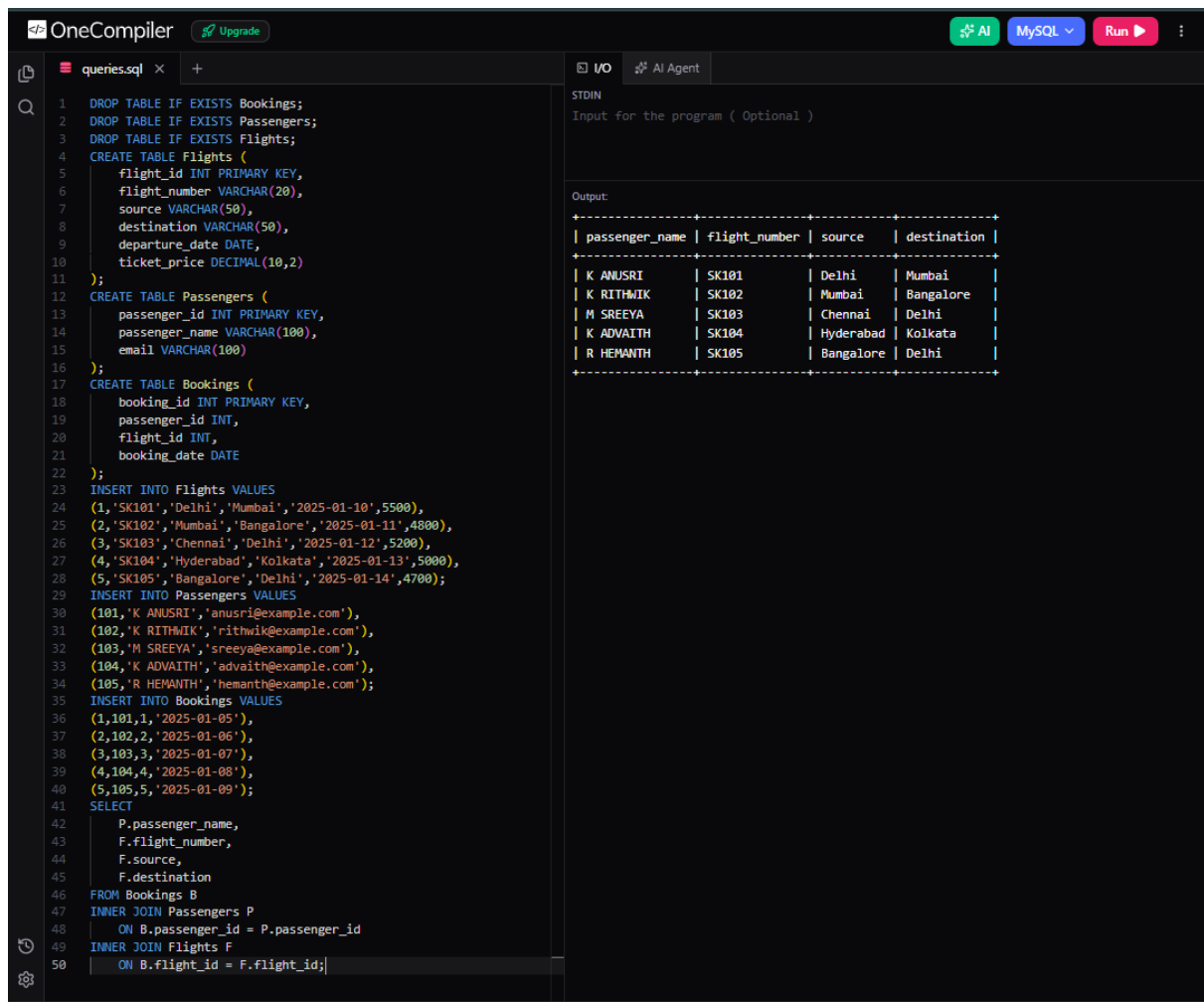
The screenshot shows the OneCompiler MySQL interface. The left pane contains the following SQL code:

```
1 DROP TABLE IF EXISTS Bookings;
2
3 CREATE TABLE Bookings (
4     booking_id INT PRIMARY KEY,
5     passenger_id INT,
6     flight_id INT,
7     booking_date DATE
8 );
9
10 INSERT INTO Bookings VALUES
11 (1, 101, 1, '2025-01-05'),
12 (2, 102, 2, '2025-01-06'),
13 (3, 103, 3, '2025-01-07'),
14 (4, 104, 4, '2025-01-08'),
15 (5, 105, 5, '2025-01-09'),
16 (6, 101, 6, '2025-01-10'),
17 (7, 102, 7, '2025-01-11'),
18 (8, 103, 8, '2025-01-12'),
19 (9, 104, 9, '2025-01-13'),
20 (10, 105, 10, '2025-01-14');
21
22 SELECT * FROM Bookings;
```

The right pane shows the output of the query, displaying a table with 10 rows of booking records:

booking_id	passenger_id	flight_id	booking_date
1	101	1	2025-01-05
2	102	2	2025-01-06
3	103	3	2025-01-07
4	104	4	2025-01-08
5	105	5	2025-01-09
6	101	6	2025-01-10
7	102	7	2025-01-11
8	103	8	2025-01-12
9	104	9	2025-01-13
10	105	10	2025-01-14

8. Display Passenger and Flight Details Using INNER JOIN



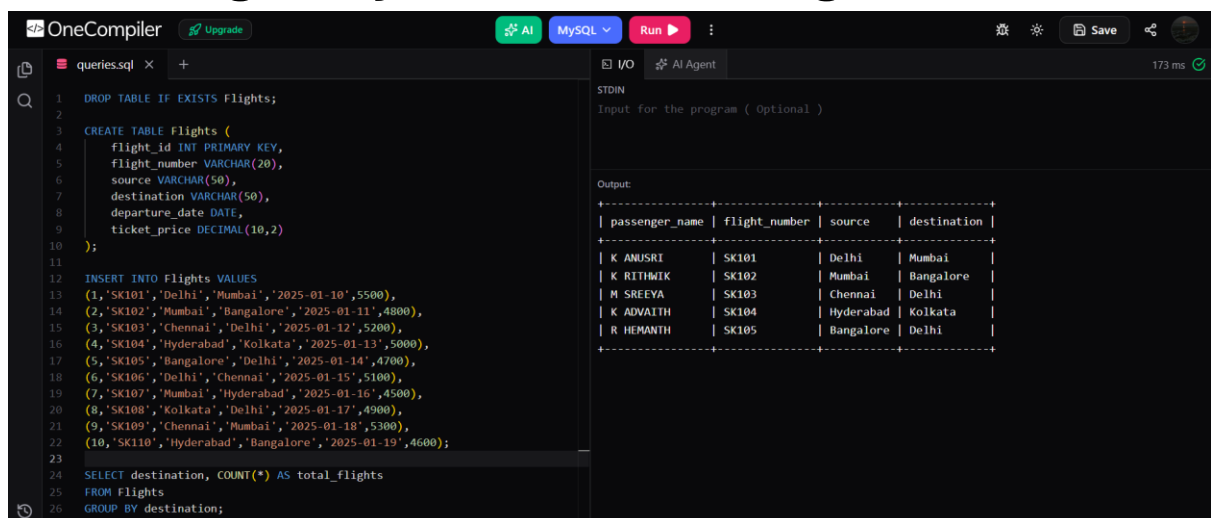
The screenshot shows the OneCompiler IDE with a MySQL database. The code in the editor creates three tables: Flights, Passengers, and Bookings. It then inserts data into these tables and finally displays the passenger and flight details using an INNER JOIN.

```
1 DROP TABLE IF EXISTS Bookings;
2 DROP TABLE IF EXISTS Passengers;
3 DROP TABLE IF EXISTS Flights;
4 CREATE TABLE Flights (
5     flight_id INT PRIMARY KEY,
6     flight_number VARCHAR(20),
7     source VARCHAR(50),
8     destination VARCHAR(50),
9     departure_date DATE,
10    ticket_price DECIMAL(10,2)
11);
12 CREATE TABLE Passengers (
13     passenger_id INT PRIMARY KEY,
14     passenger_name VARCHAR(100),
15     email VARCHAR(100)
16);
17 CREATE TABLE Bookings (
18     booking_id INT PRIMARY KEY,
19     passenger_id INT,
20     flight_id INT,
21     booking_date DATE
22);
23 INSERT INTO Flights VALUES
24 (1,'SK101','Delhi','Mumbai','2025-01-10',5500),
25 (2,'SK102','Mumbai','Bangalore','2025-01-11',4800),
26 (3,'SK103','Chennai','Delhi','2025-01-12',5200),
27 (4,'SK104','Hyderabad','Kolkata','2025-01-13',5000),
28 (5,'SK105','Bangalore','Delhi','2025-01-14',4700);
29 INSERT INTO Passengers VALUES
30 (101,'K ANUSRI','anusri@example.com'),
31 (102,'K RITHVIK','rithvik@example.com'),
32 (103,'M SREEYA','sreeya@example.com'),
33 (104,'K ADVAITH','advait@example.com'),
34 (105,'R HEMANTH','hemanth@example.com');
35 INSERT INTO Bookings VALUES
36 (1,101,1,'2025-01-05'),
37 (2,102,2,'2025-01-06'),
38 (3,103,3,'2025-01-07'),
39 (4,104,4,'2025-01-08'),
40 (5,105,5,'2025-01-09');
41 SELECT
42     P.passenger_name,
43     F.flight_number,
44     F.source,
45     F.destination
46 FROM Bookings B
47 INNER JOIN Passengers P
48     ON B.passenger_id = P.passenger_id
49 INNER JOIN Flights F
50     ON B.flight_id = F.flight_id;
```

The output shows the following data:

passenger_name	flight_number	source	destination
K ANUSRI	SK101	Delhi	Mumbai
K RITHVIK	SK102	Mumbai	Bangalore
M SREEYA	SK103	Chennai	Delhi
K ADVAITH	SK104	Hyderabad	Kolkata
R HEMANTH	SK105	Bangalore	Delhi

9. Count Flights by Destination Using GROUP BY



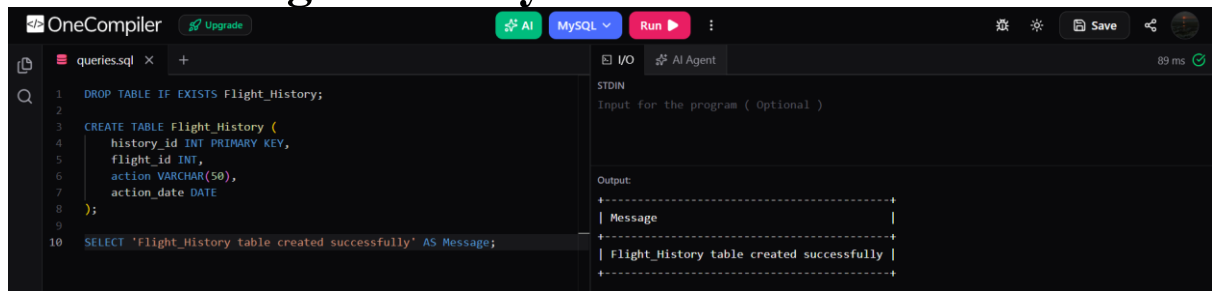
The screenshot shows the OneCompiler IDE with a MySQL database. The code in the editor creates the Flights table, inserts data, and then displays the count of flights by destination using a GROUP BY clause.

```
1 DROP TABLE IF EXISTS Flights;
2
3 CREATE TABLE Flights (
4     flight_id INT PRIMARY KEY,
5     flight_number VARCHAR(20),
6     source VARCHAR(50),
7     destination VARCHAR(50),
8     departure_date DATE,
9     ticket_price DECIMAL(10,2)
10);
11
12 INSERT INTO Flights VALUES
13 (1,'SK101','Delhi','Mumbai','2025-01-10',5500),
14 (2,'SK102','Mumbai','Bangalore','2025-01-11',4800),
15 (3,'SK103','Chennai','Delhi','2025-01-12',5200),
16 (4,'SK104','Hyderabad','Kolkata','2025-01-13',5000),
17 (5,'SK105','Bangalore','Delhi','2025-01-14',4700),
18 (6,'SK106','Delhi','Chennai','2025-01-15',5100),
19 (7,'SK107','Mumbai','Hyderabad','2025-01-16',4500),
20 (8,'SK108','Kolkata','Delhi','2025-01-17',4900),
21 (9,'SK109','Chennai','Mumbai','2025-01-18',5300),
22 (10,'SK110','Hyderabad','Bangalore','2025-01-19',4600);
23
24 SELECT destination, COUNT(*) AS total_flights
25 FROM Flights
26 GROUP BY destination;
```

The output shows the following data:

passenger_name	flight_number	source	destination
K ANUSRI	SK101	Delhi	Mumbai
K RITHVIK	SK102	Mumbai	Bangalore
M SREEYA	SK103	Chennai	Delhi
K ADVAITH	SK104	Hyderabad	Kolkata
R HEMANTH	SK105	Bangalore	Delhi

10. Create Flight History Table

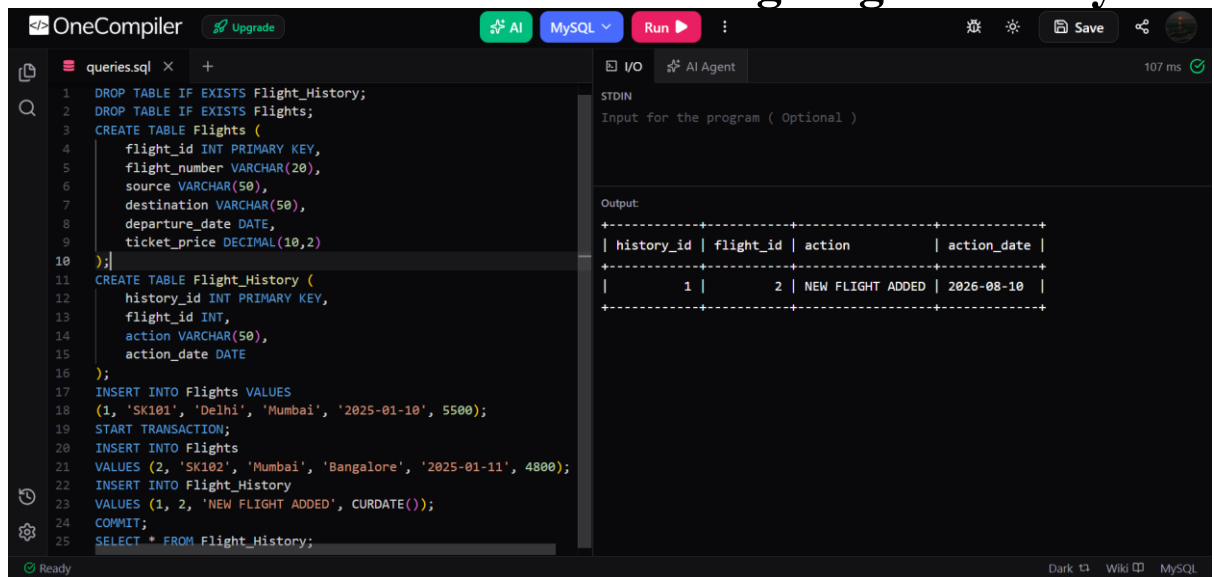


The screenshot shows the OneCompiler interface with a MySQL database. The code in the editor creates a table named Flight_History with columns: history_id (INT, PRIMARY KEY), flight_id (INT), action (VARCHAR(50)), and action_date (DATE). The output shows a message: "Flight_History table created successfully".

```
1 DROP TABLE IF EXISTS Flight_History;
2
3 CREATE TABLE Flight_History (
4   history_id INT PRIMARY KEY,
5   flight_id INT,
6   action VARCHAR(50),
7   action_date DATE
8 );
9
10 SELECT 'Flight_History table created successfully' AS Message;
```

Message
Flight_History table created successfully

11. Demonstrate Transaction Using Flight History

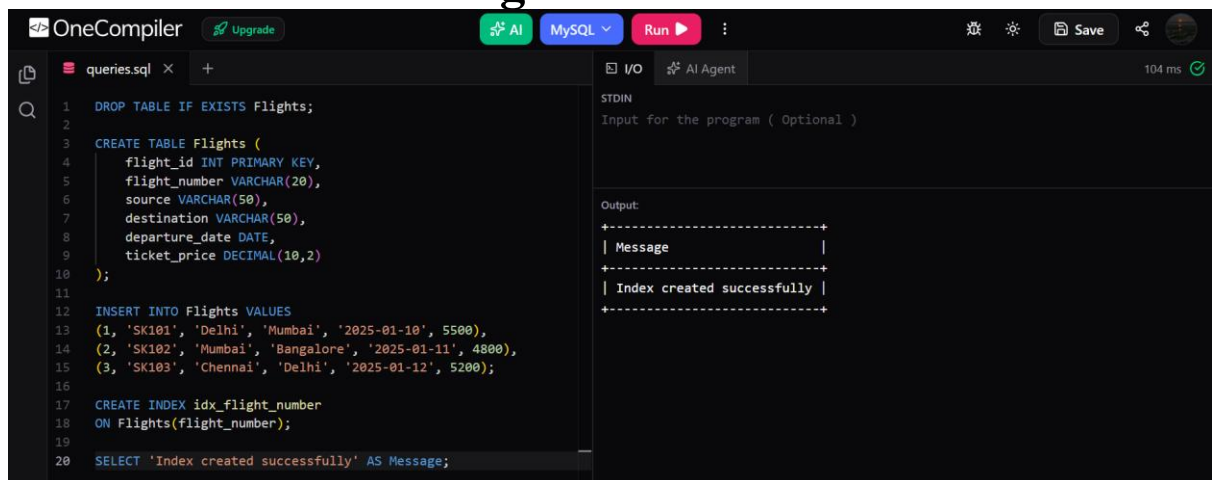


The screenshot shows the OneCompiler interface with a MySQL database. The code creates two tables: Flights and Flight_History. It then inserts data into Flights and Flight_History, demonstrating a transaction. The output shows a message: "NEW FLIGHT ADDED".

```
1 DROP TABLE IF EXISTS Flight_History;
2 DROP TABLE IF EXISTS Flights;
3 CREATE TABLE Flights (
4   flight_id INT PRIMARY KEY,
5   flight_number VARCHAR(20),
6   source VARCHAR(50),
7   destination VARCHAR(50),
8   departure_date DATE,
9   ticket_price DECIMAL(10,2)
10 );
11 CREATE TABLE Flight_History (
12   history_id INT PRIMARY KEY,
13   flight_id INT,
14   action VARCHAR(50),
15   action_date DATE
16 );
17 INSERT INTO Flights VALUES
18 (1, 'SK101', 'Delhi', 'Mumbai', '2025-01-10', 5500);
19 START TRANSACTION;
20 INSERT INTO Flights
21 VALUES (2, 'SK102', 'Mumbai', 'Bangalore', '2025-01-11', 4800);
22 INSERT INTO Flight_History
23 VALUES (1, 2, 'NEW FLIGHT ADDED', CURDATE());
24 COMMIT;
25 SELECT * FROM Flight_History;
```

history_id	flight_id	action	action_date
1	2	NEW FLIGHT ADDED	2026-08-10

12. Create Index on Flight Number

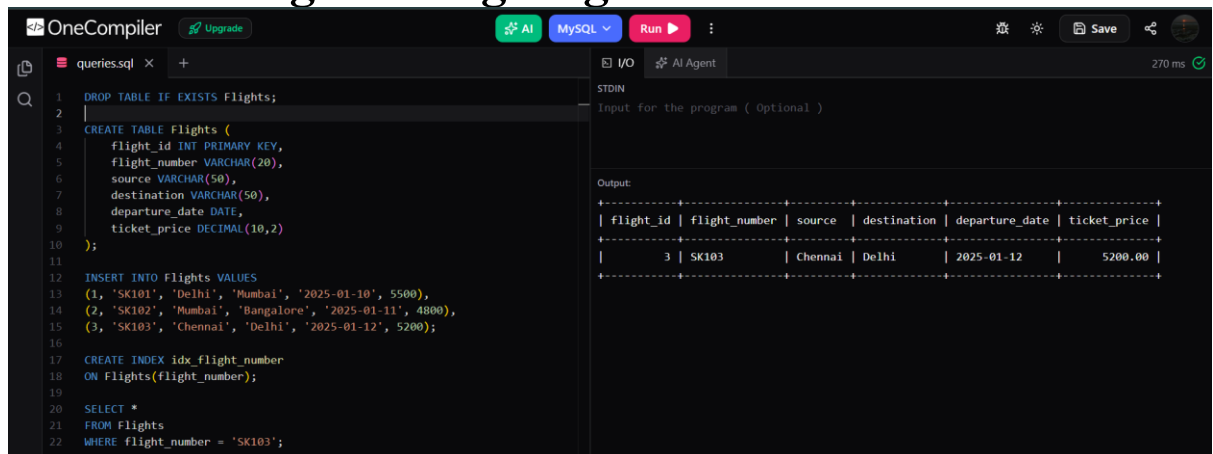


The screenshot shows the OneCompiler interface with a MySQL database. The code creates a table named Flights with columns: flight_id (INT, PRIMARY KEY), flight_number (VARCHAR(20)), source (VARCHAR(50)), destination (VARCHAR(50)), departure_date (DATE), and ticket_price (DECIMAL(10,2)). It then creates an index on the flight_number column. The output shows a message: "Index created successfully".

```
1 DROP TABLE IF EXISTS Flights;
2
3 CREATE TABLE Flights (
4   flight_id INT PRIMARY KEY,
5   flight_number VARCHAR(20),
6   source VARCHAR(50),
7   destination VARCHAR(50),
8   departure_date DATE,
9   ticket_price DECIMAL(10,2)
10 );
11
12 INSERT INTO Flights VALUES
13 (1, 'SK101', 'Delhi', 'Mumbai', '2025-01-10', 5500),
14 (2, 'SK102', 'Mumbai', 'Bangalore', '2025-01-11', 4800),
15 (3, 'SK103', 'Chennai', 'Delhi', '2025-01-12', 5200);
16
17 CREATE INDEX idx_flight_number
18 ON Flights(flight_number);
19
20 SELECT 'Index created successfully' AS Message;
```

Message
Index created successfully

13. Search Flight Using Flight Number



The screenshot shows the OneCompiler MySQL interface. The left pane contains the following SQL code:

```
1 DROP TABLE IF EXISTS Flights;
2
3 CREATE TABLE Flights (
4     flight_id INT PRIMARY KEY,
5     flight_number VARCHAR(20),
6     source VARCHAR(50),
7     destination VARCHAR(50),
8     departure_date DATE,
9     ticket_price DECIMAL(10,2)
10 );
11
12 INSERT INTO Flights VALUES
13 (1, 'SK101', 'Delhi', 'Mumbai', '2025-01-10', 5500),
14 (2, 'SK102', 'Mumbai', 'Bangalore', '2025-01-11', 4800),
15 (3, 'SK103', 'Chennai', 'Delhi', '2025-01-12', 5200);
16
17 CREATE INDEX idx_flight_number
18 ON Flights(flight_number);
19
20 SELECT *
21 FROM Flights
22 WHERE flight_number = 'SK103';
```

The right pane shows the output of the query, which is a table with 6 columns: flight_id, flight_number, source, destination, departure_date, and ticket_price. The output shows one row for flight SK103.

flight_id	flight_number	source	destination	departure_date	ticket_price
3	SK103	Chennai	Delhi	2025-01-12	5200.00

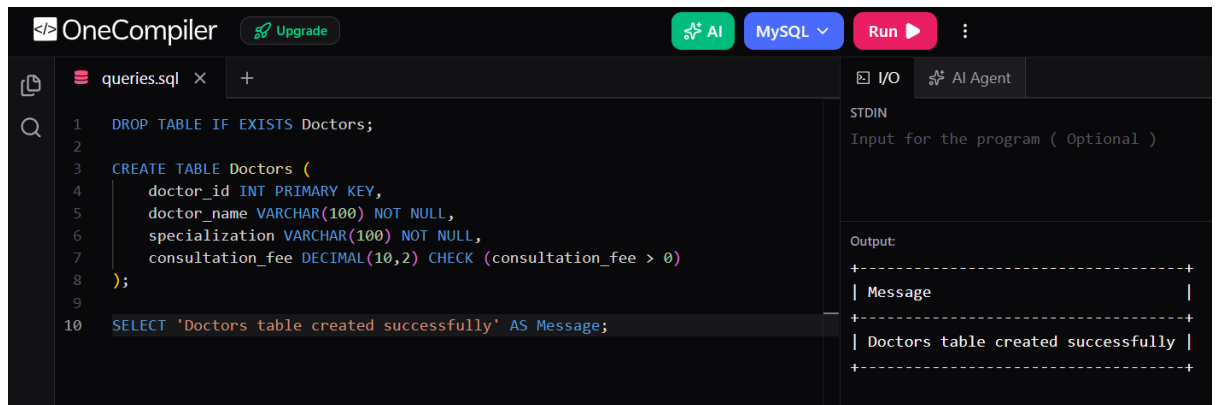
ASSIGNMENT – 2

1. Create Database

CREATE DATABASE IF NOT EXISTS skytrack_db;

USE skytrack_db;

2. Create Doctors Table

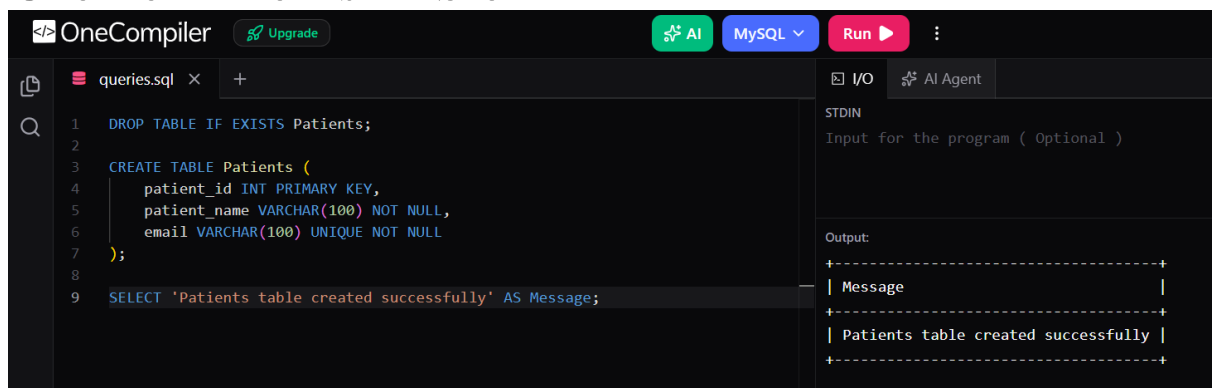


The screenshot shows the OneCompiler interface with a MySQL database selected. The code in the editor is as follows:

```
1 DROP TABLE IF EXISTS Doctors;
2
3 CREATE TABLE Doctors (
4     doctor_id INT PRIMARY KEY,
5     doctor_name VARCHAR(100) NOT NULL,
6     specialization VARCHAR(100) NOT NULL,
7     consultation_fee DECIMAL(10,2) CHECK (consultation_fee > 0)
8 );
9
10 SELECT 'Doctors table created successfully' AS Message;
```

The output on the right shows the message: "Doctors table created successfully".

3. Create Patients Table

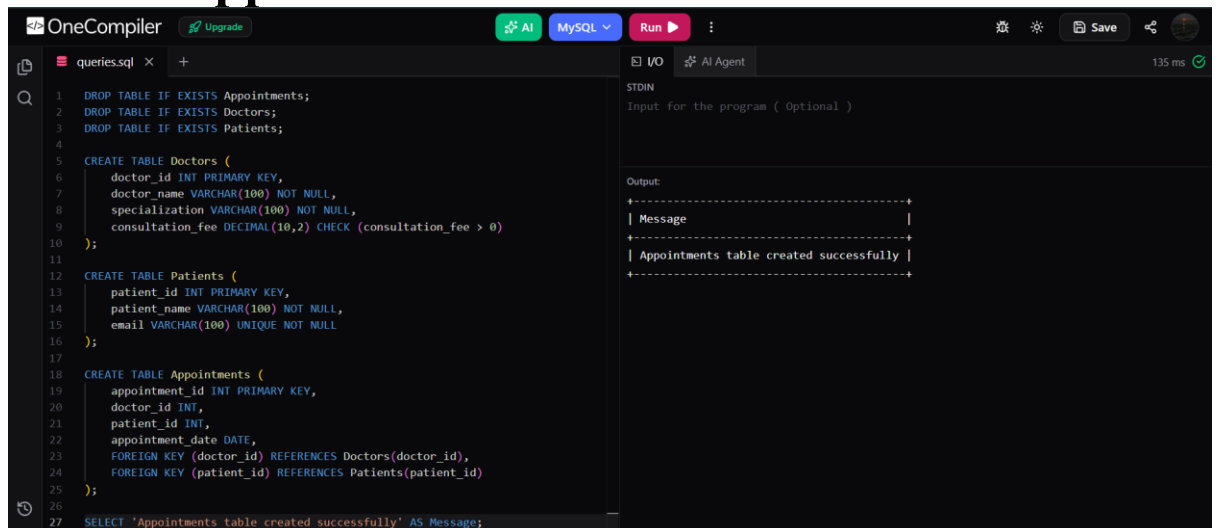


The screenshot shows the OneCompiler interface with a MySQL database selected. The code in the editor is as follows:

```
1 DROP TABLE IF EXISTS Patients;
2
3 CREATE TABLE Patients (
4     patient_id INT PRIMARY KEY,
5     patient_name VARCHAR(100) NOT NULL,
6     email VARCHAR(100) UNIQUE NOT NULL
7 );
8
9 SELECT 'Patients table created successfully' AS Message;
```

The output on the right shows the message: "Patients table created successfully".

4. Create Appointments Table

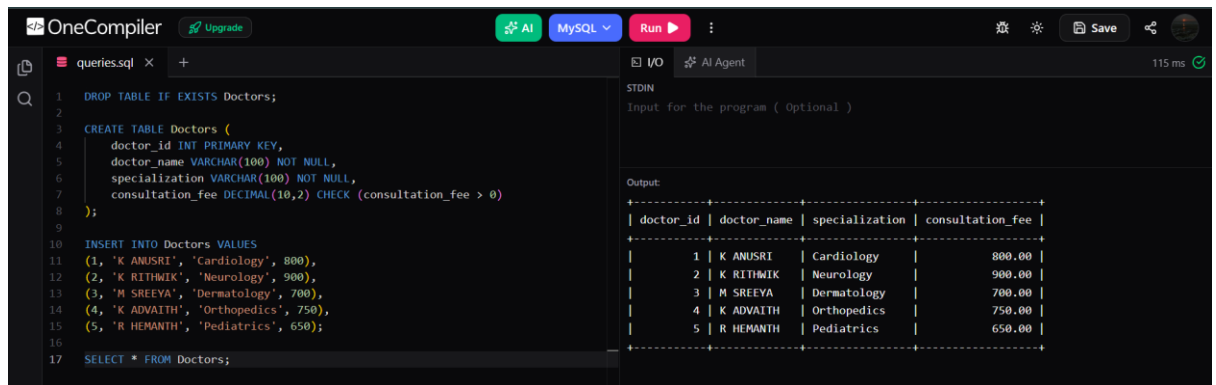


The screenshot shows the OneCompiler interface with a MySQL database selected. The code in the editor is as follows:

```
1 DROP TABLE IF EXISTS Appointments;
2 DROP TABLE IF EXISTS Doctors;
3 DROP TABLE IF EXISTS Patients;
4
5 CREATE TABLE Doctors (
6     doctor_id INT PRIMARY KEY,
7     doctor_name VARCHAR(100) NOT NULL,
8     specialization VARCHAR(100) NOT NULL,
9     consultation_fee DECIMAL(10,2) CHECK (consultation_fee > 0)
10 );
11
12 CREATE TABLE Patients (
13     patient_id INT PRIMARY KEY,
14     patient_name VARCHAR(100) NOT NULL,
15     email VARCHAR(100) UNIQUE NOT NULL
16 );
17
18 CREATE TABLE Appointments (
19     appointment_id INT PRIMARY KEY,
20     doctor_id INT,
21     patient_id INT,
22     appointment_date DATE,
23     FOREIGN KEY (doctor_id) REFERENCES Doctors(doctor_id),
24     FOREIGN KEY (patient_id) REFERENCES Patients(patient_id)
25 );
26
27 SELECT 'Appointments table created successfully' AS Message;
```

The output on the right shows the message: "Appointments table created successfully".

5. Insert Doctor Records

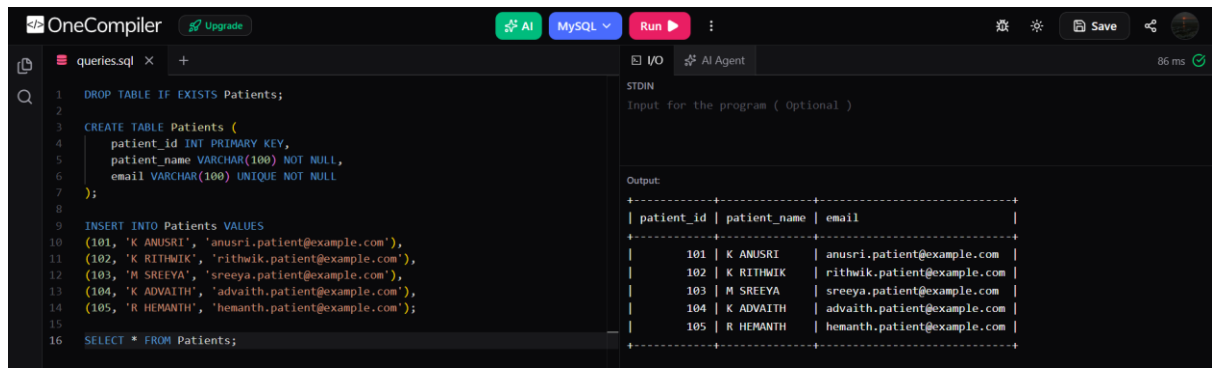


```
1 DROP TABLE IF EXISTS Doctors;
2
3 CREATE TABLE Doctors (
4     doctor_id INT PRIMARY KEY,
5     doctor_name VARCHAR(100) NOT NULL,
6     specialization VARCHAR(100) NOT NULL,
7     consultation_fee DECIMAL(10,2) CHECK (consultation_fee > 0)
8 );
9
10 INSERT INTO Doctors VALUES
11 (1, 'K ANUSRI', 'Cardiology', 800),
12 (2, 'K RITHWIK', 'Neurology', 900),
13 (3, 'M SREEYA', 'Dermatology', 700),
14 (4, 'K ADVAITH', 'Orthopedics', 750),
15 (5, 'R HEMANTH', 'Pediatrics', 650);
16
17 SELECT * FROM Doctors;
```

Output:

doctor_id	doctor_name	specialization	consultation_fee
1	K ANUSRI	Cardiology	800.00
2	K RITHWIK	Neurology	900.00
3	M SREEYA	Dermatology	700.00
4	K ADVAITH	Orthopedics	750.00
5	R HEMANTH	Pediatrics	650.00

6. Insert Patient Records

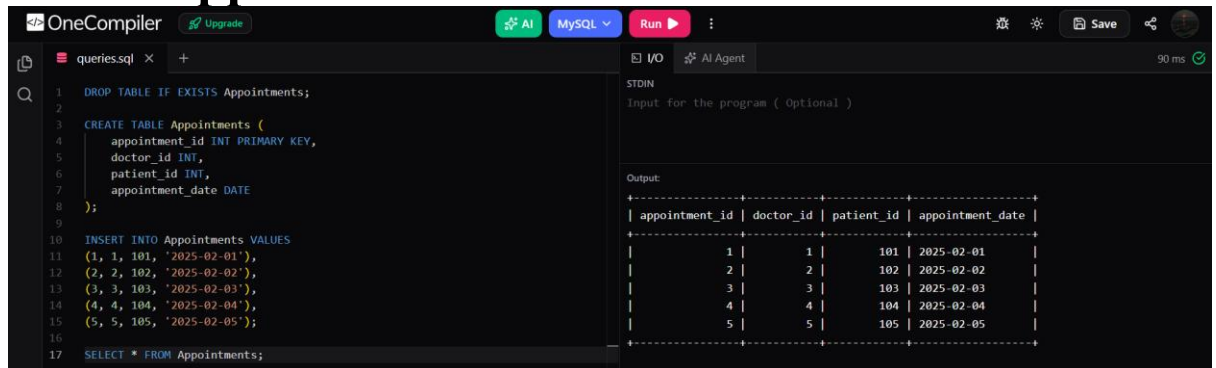


```
1 DROP TABLE IF EXISTS Patients;
2
3 CREATE TABLE Patients (
4     patient_id INT PRIMARY KEY,
5     patient_name VARCHAR(100) NOT NULL,
6     email VARCHAR(100) UNIQUE NOT NULL
7 );
8
9 INSERT INTO Patients VALUES
10 (101, 'K ANUSRI', 'anusri.patient@example.com'),
11 (102, 'K RITHWIK', 'rithwik.patient@example.com'),
12 (103, 'M SREEYA', 'sreeya.patient@example.com'),
13 (104, 'K ADVAITH', 'advaith.patient@example.com'),
14 (105, 'R HEMANTH', 'hemanth.patient@example.com');
15
16 SELECT * FROM Patients;
```

Output:

patient_id	patient_name	email
101	K ANUSRI	anusri.patient@example.com
102	K RITHWIK	rithwik.patient@example.com
103	M SREEYA	sreeya.patient@example.com
104	K ADVAITH	advaith.patient@example.com
105	R HEMANTH	hemanth.patient@example.com

7. Insert Appointment Records



```
1 DROP TABLE IF EXISTS Appointments;
2
3 CREATE TABLE Appointments (
4     appointment_id INT PRIMARY KEY,
5     doctor_id INT,
6     patient_id INT,
7     appointment_date DATE
8 );
9
10 INSERT INTO Appointments VALUES
11 (1, 1, 101, '2025-02-01'),
12 (2, 2, 102, '2025-02-02'),
13 (3, 3, 103, '2025-02-03'),
14 (4, 4, 104, '2025-02-04'),
15 (5, 5, 105, '2025-02-05');
16
17 SELECT * FROM Appointments;
```

Output:

appointment_id	doctor_id	patient_id	appointment_date
1	1	101	2025-02-01
2	2	102	2025-02-02
3	3	103	2025-02-03
4	4	104	2025-02-04
5	5	105	2025-02-05

8. Display Patient and Doctor Details Using INNER JOIN

OneCompiler Upgrade

queries.sql +

I/O AI Agent

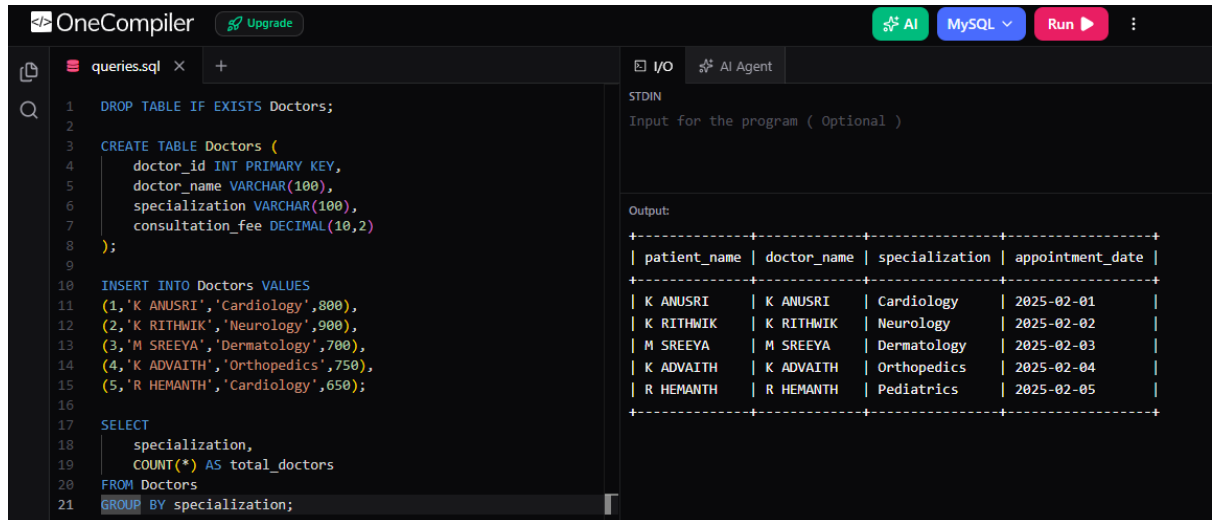
STDIN
Input for the program (Optional)

Output:

patient_name	doctor_name	specialization	appointment_date
K ANUSRI	K ANUSRI	Cardiology	2025-02-01
K RITHWIK	K RITHWIK	Neurology	2025-02-02
M SREEYA	M SREEYA	Dermatology	2025-02-03
K ADVAITH	K ADVAITH	Orthopedics	2025-02-04
R HEMANTH	R HEMANTH	Pediatrics	2025-02-05

```
1 DROP TABLE IF EXISTS Appointments;
2 DROP TABLE IF EXISTS Doctors;
3 DROP TABLE IF EXISTS Patients;
4 CREATE TABLE Doctors (
5     doctor_id INT PRIMARY KEY,
6     doctor_name VARCHAR(100),
7     specialization VARCHAR(100),
8     consultation_fee DECIMAL(10,2)
9 );
10 CREATE TABLE Patients (
11     patient_id INT PRIMARY KEY,
12     patient_name VARCHAR(100),
13     email VARCHAR(100)
14 );
15 CREATE TABLE Appointments (
16     appointment_id INT PRIMARY KEY,
17     doctor_id INT,
18     patient_id INT,
19     appointment_date DATE
20 );
21 INSERT INTO Doctors VALUES
22 (1,'K ANUSRI','Cardiology',800),
23 (2,'K RITHWIK','Neurology',900),
24 (3,'M SREEYA','Dermatology',700),
25 (4,'K ADVAITH','Orthopedics',750),
26 (5,'R HEMANTH','Pediatrics',650);
27 INSERT INTO Patients VALUES
28 (101,'K ANUSRI','anusri.patient@example.com'),
29 (102,'K RITHWIK','rithwik.patient@example.com'),
30 (103,'M SREEYA','sreeya.patient@example.com'),
31 (104,'K ADVAITH','advaith.patient@example.com'),
32 (105,'R HEMANTH','hemanth.patient@example.com');
33 INSERT INTO Appointments VALUES
34 (1,1,101,'2025-02-01'),
35 (2,2,102,'2025-02-02'),
36 (3,3,103,'2025-02-03'),
37 (4,4,104,'2025-02-04'),
38 (5,5,105,'2025-02-05');
39 SELECT
40     P.patient_name,
41     D.doctor_name,
42     D.specialization,
43     A.appointment_date
44 FROM Appointments A
45 INNER JOIN Patients P
46     ON A.patient_id = P.patient_id
47 INNER JOIN Doctors D
48     ON A.doctor_id = D.doctor_id;
```

9. Count Doctors by Specialization Using GROUP BY

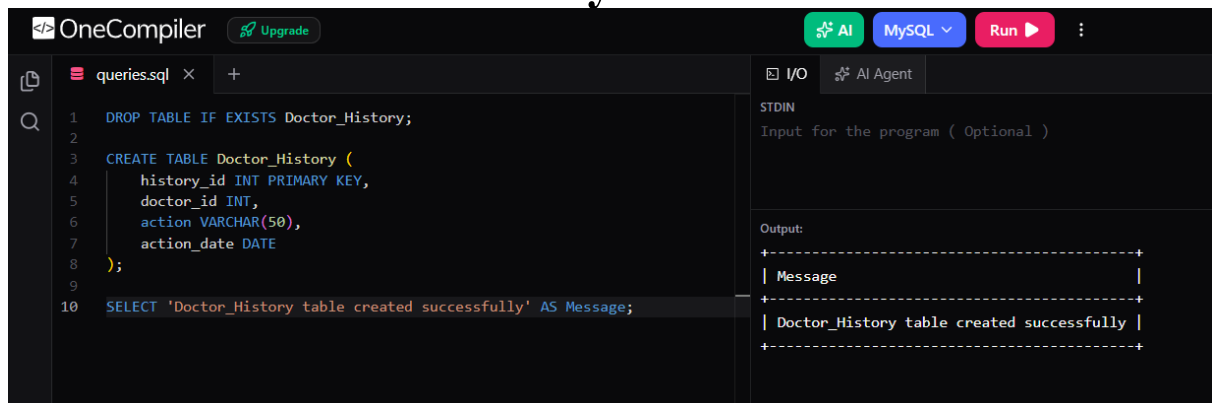


```
1 DROP TABLE IF EXISTS Doctors;
2
3 CREATE TABLE Doctors (
4     doctor_id INT PRIMARY KEY,
5     doctor_name VARCHAR(100),
6     specialization VARCHAR(100),
7     consultation_fee DECIMAL(10,2)
8 );
9
10 INSERT INTO Doctors VALUES
11 (1,'K ANUSRI','Cardiology',800),
12 (2,'K RITHWIK','Neurology',900),
13 (3,'M SREEYA','Dermatology',700),
14 (4,'K ADVAITH','Orthopedics',750),
15 (5,'R HEMANTH','Cardiology',650);
16
17 SELECT
18     specialization,
19     COUNT(*) AS total_doctors
20 FROM Doctors
21 GROUP BY specialization;
```

Output:

patient_name	doctor_name	specialization	appointment_date
K ANUSRI	K ANUSRI	Cardiology	2025-02-01
K RITHWIK	K RITHWIK	Neurology	2025-02-02
M SREEYA	M SREEYA	Dermatology	2025-02-03
K ADVAITH	K ADVAITH	Orthopedics	2025-02-04
R HEMANTH	R HEMANTH	Pediatrics	2025-02-05

10. Create Doctor History Table

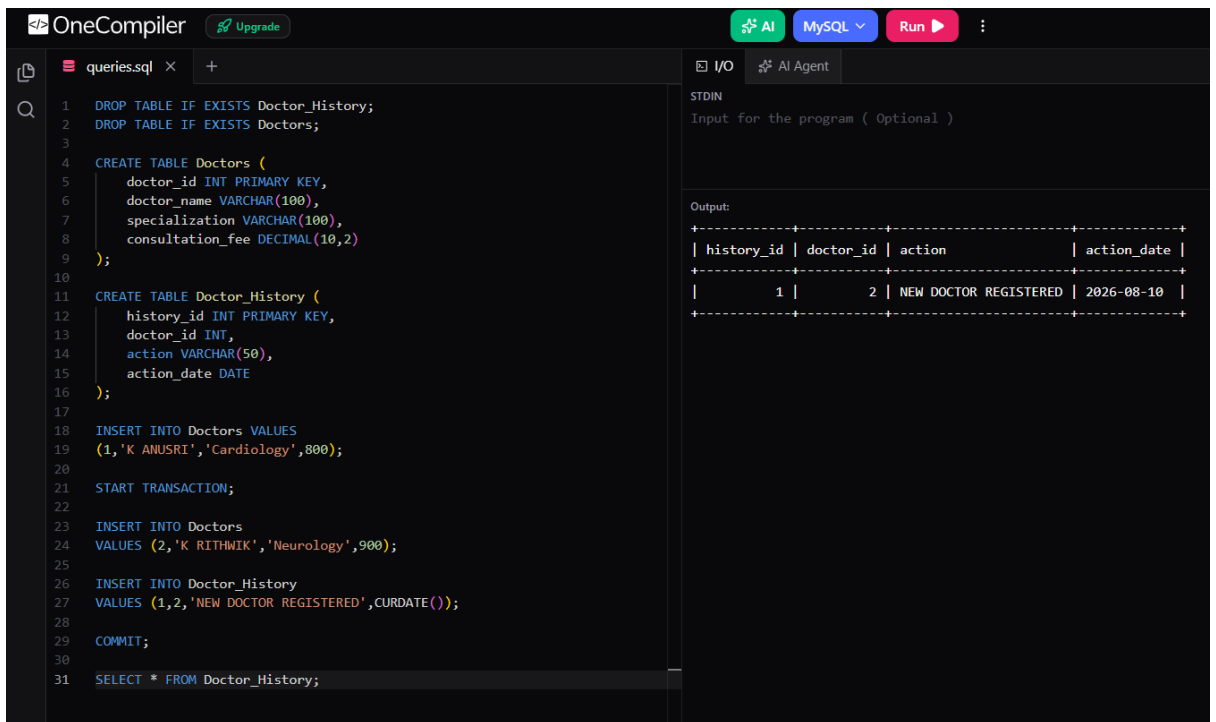


```
1 DROP TABLE IF EXISTS Doctor_History;
2
3 CREATE TABLE Doctor_History (
4     history_id INT PRIMARY KEY,
5     doctor_id INT,
6     action VARCHAR(50),
7     action_date DATE
8 );
9
10 SELECT 'Doctor_History table created successfully' AS Message;
```

Output:

Message
Doctor_History table created successfully

11. Demonstrate Transaction Using Doctor_History



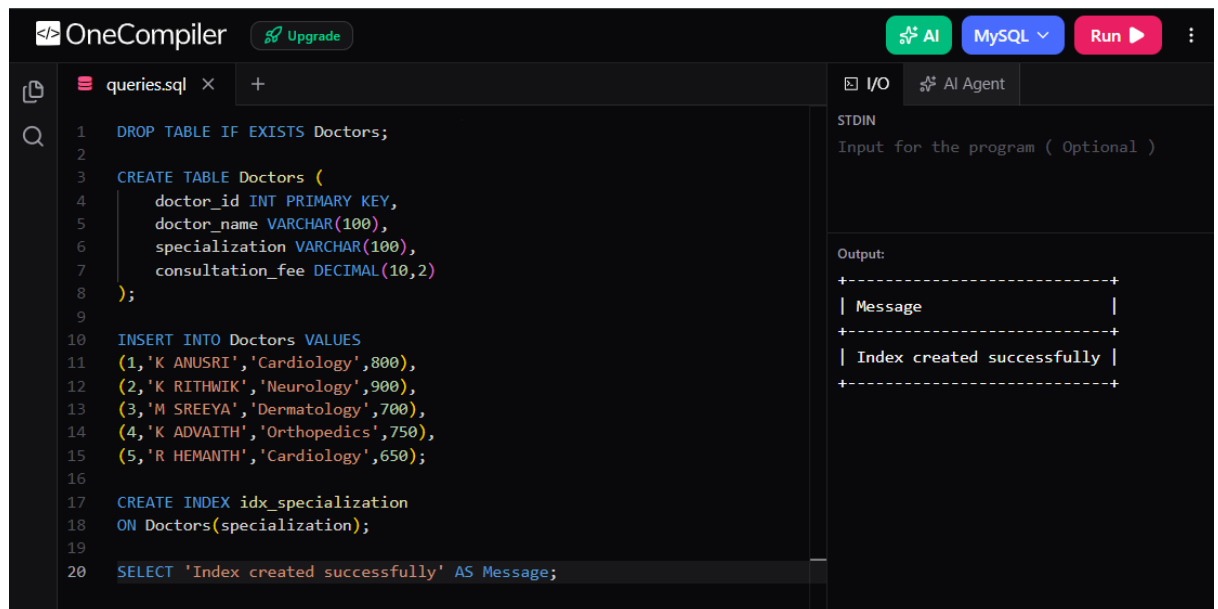
The screenshot shows the OneCompiler MySQL interface. The left pane contains a SQL script in `queries.sql` with line numbers 1 through 31. The script creates two tables: `Doctors` and `Doctor_History`. It inserts two doctors into the `Doctors` table and then inserts a record into the `Doctor_History` table within a transaction. The right pane shows the output of the query `SELECT * FROM Doctor_History;`, displaying a single row with the history of the first doctor's registration.

```
1 DROP TABLE IF EXISTS Doctor_History;
2 DROP TABLE IF EXISTS Doctors;
3
4 CREATE TABLE Doctors (
5     doctor_id INT PRIMARY KEY,
6     doctor_name VARCHAR(100),
7     specialization VARCHAR(100),
8     consultation_fee DECIMAL(10,2)
9 );
10
11 CREATE TABLE Doctor_History (
12     history_id INT PRIMARY KEY,
13     doctor_id INT,
14     action VARCHAR(50),
15     action_date DATE
16 );
17
18 INSERT INTO Doctors VALUES
19 (1,'K ANUSRI','Cardiology',800);
20
21 START TRANSACTION;
22
23 INSERT INTO Doctors
24 VALUES (2,'K RITHWIK','Neurology',900);
25
26 INSERT INTO Doctor_History
27 VALUES (1,2,'NEW DOCTOR REGISTERED',CURDATE());
28
29 COMMIT;
30
31 SELECT * FROM Doctor_History;
```

Output:

history_id	doctor_id	action	action_date
1	2	NEW DOCTOR REGISTERED	2026-08-10

12. Create Index on Doctor Specialization



The screenshot shows the OneCompiler MySQL interface. The left pane contains a SQL script in `queries.sql` with line numbers 1 through 20. The script drops the `Doctors` table, creates it with five doctors, and then creates an index on the `specialization` column. The right pane shows the output of the query `SELECT 'Index created successfully' AS Message;`, displaying the message "Index created successfully".

```
1 DROP TABLE IF EXISTS Doctors;
2
3 CREATE TABLE Doctors (
4     doctor_id INT PRIMARY KEY,
5     doctor_name VARCHAR(100),
6     specialization VARCHAR(100),
7     consultation_fee DECIMAL(10,2)
8 );
9
10 INSERT INTO Doctors VALUES
11 (1,'K ANUSRI','Cardiology',800),
12 (2,'K RITHWIK','Neurology',900),
13 (3,'M SREEYA','Dermatology',700),
14 (4,'K ADVAITH','Orthopedics',750),
15 (5,'R HEMANTH','Cardiology',650);
16
17 CREATE INDEX idx_specialization
18 ON Doctors(specialization);
19
20 SELECT 'Index created successfully' AS Message;
```

Output:

Message
Index created successfully

13. Search Doctors Using Specialization Index

OneCompiler Upgrade

AI MySQL Run

queries.sql

```
1 DROP TABLE IF EXISTS Doctors;
2
3 CREATE TABLE Doctors (
4     doctor_id INT PRIMARY KEY,
5     doctor_name VARCHAR(100),
6     specialization VARCHAR(100),
7     consultation_fee DECIMAL(10,2)
8 );
9
10 INSERT INTO Doctors VALUES
11 (1,'K ANUSRI','Cardiology',800),
12 (2,'K RITHWIK','Neurology',900),
13 (3,'M SREEYA','Dermatology',700),
14 (4,'K ADVAITH','Orthopedics',750),
15 (5,'R HEMANTH','Cardiology',650);
16
17 CREATE INDEX idx_specialization
18 ON Doctors(specialization);
19
20 SELECT *
21 FROM Doctors
22 WHERE specialization = 'Cardiology';
```

I/O AI Agent

STDIN
Input for the program (Optional)

Output:

doctor_id	doctor_name	specialization	consultation_fee
1	K ANUSRI	Cardiology	800.00
5	R HEMANTH	Cardiology	650.00